

Modos de permiso en Claude Code

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES Sesión 5 de 12 · Notas en español (es-ES)

La sesión 5 (12 de agosto de 2026) reanudó la generación frontend interrumpida y abrió con un repaso de los **modos de permiso**: cuánto puede hacer Claude Code sin parar a preguntaros. El formador enumeró cuatro comportamientos que debéis contrastar por escrito: pedir permiso para cada actividad, aceptar ediciones automáticamente, planificar y luego ejecutar, y auto / ejecución «sin parar». Las cadenas exactas de la UI cambian; la superficie de control no.

FIAROS DE LA DOCUMENTACIÓN VIGENTE PARA LAS ETIQUETAS

La grabación dice «ask», «edit automatically», «plan» y «auto decide / no stop». Los docs actuales pueden usar default, accept edits, plan y bypass permissions. Para el examen, describid *qué se autoaprueba* y *cuándo aún podéis editar la lista de to-dos*. Empezad por la [visión general de Claude Code](#).

Por qué existen los modos

Claude Code es un agente que puede editar ficheros, lanzar bash, llamar tools MCP y borrar rutas. Cada una es un *tool use* que en modo por defecto quiere consentimiento. Los modos son la política de ese consentimiento: con qué frecuencia el bucle se detiene. No son otro modelo ni sustituyen a CLAUDE.md. Tampoco son «modelos inseguros frente a seguros»: Opus en auto sigue siendo auto.

Los cuatro comportamientos de clase

Nombre en el aula	Qué puede hacer Claude sin preguntar	Cuándo usarlo	Riesgo principal
Ask (todas las actividades)	Poco: cada edición, comando de shell y llamada MCP puede pedir confirmación.	Aprender la herramienta; repos desconocidos; secretos de producción cerca.	Lento; vosotros sois el cuello de botella.
Edit automatically (accept edits)	Escrituras/ediciones de ficheros; probablemente seguirá preguntando por shell o red arriesgados.	Os fiáis del plan y queréis menos clics de Save/Apply.	Ediciones malas silenciosas si no miráis el git diff.
Plan	Produce un to-do / plan primero; lo revisáis y editáis; la ejecución empieza solo al aceptar.	Generación en varios pasos, elegir Docker frente a Playwright, cualquier cosa que queráis reescribir.	Una vez arranca la ejecución, el formador dijo que no podéis editar ese plan en vuelo.
Auto / no stop (estilo bypass)	Los prompts de permiso se resuelven solos; el bucle no espera.	Generación larga desatendida cuando ya hay hooks y listas allow/deny.	Bash destructivo (rm, git restore, etc.) salvo que hooks previos a la tool lo bloqueen (módulo 06).

El modo plan es el que se cruza con los agentes

Un alumno preguntó si aún se puede intervenir si los agentes corren sin interrupción. El formador separó dos ideas: el **plan del orquestador** (la lista de to-dos: andamiaje Vite, tipos compartidos según OpenAPI, páginas, tests...) es editable *en modo plan, antes de execute*. Si Playwright está en la lista y queréis Selenium, cambiad el plan y luego dejad correr a los agentes. Cuando execute ha empezado, observáis, omitís o paráis: no reescribís en silencio el plan congelado. El humano en el bucle se concentra al principio, en la planificación, no en cada turno de subagente.

- Plan mode también aparece en escenarios de examen posteriores («plan mode and direct execution»). Direct execution es lo contrario: saltarse el tablero de to-dos revisable y seguir.
- Los subagentes pueden llevar su propio modo de permiso (p. ej. bypass para que un revisor de código no se bloquee en cada grep). Eso se configura en el markdown del agente, no solo en el selector global (véase M05-C03).
- El modo auto es exactamente cuando los hooks del módulo 06 (bloquear comandos peligrosos, proteger .env) dejan de ser opcionales.

El consentimiento sigue siendo vuestro

Incluso en ask, vale el descargo de la sesión 4: los comandos se ejecutan con el consentimiento del usuario final. Pasar a auto no mueve la responsabilidad legal o de empresa a Anthropic. Las normas de aula UST siguen prohibiendo pegar secretos de cliente; el modo auto escribirá lo que el modelo proponga en el árbol de trabajo si no habéis bloqueado esas rutas.

AUTO SIN HOOKS

La sesión del 13 de agosto mostró un rm de un fichero de prueba con auto activo; un hook previo debía denegarlo. Hasta que ese hook esté probado en la *misma* superficie (terminal frente a chat del IDE), no dejéis auto en un ordenador que alcance credenciales de producción. Recargad VS Code / webviews tras cambiar settings.json.

Cómo se vio al terminar la app

Continuar la generación desde el chat anterior funcionó porque la sesión aún tenía la lista de to-dos (contexto + plan), no porque se reenviara un prompt gigante. El modo de permiso decide si cada to-do restante se detiene. Si la generación «tarda muchísimo», suele ser el bucle agentic y la verificación, no un diálogo de permiso atascado — salvo que estéis en ask y se os haya pasado un prompt en otro panel.

Comprobación de estudio

Estad preparados para mapear un escenario: «el desarrollador quiere revisar pasos de arquitectura pero no cada escritura de fichero» → plan y luego accept-edits, no auto total. «Refactor desatendido tipo CI en una rama desechable con hooks de protect-files» → auto. «Primer día con Claude Code» → ask. Escribid el comportamiento y mencionad que el nombre del interruptor hay que comprobarlo en los docs.

Documentación oficial

- **Visión general de Claude Code**

<https://code.claude.com/docs/en/overview>

Modos de permiso, plan mode y cómo el agente pide tools.

- **Visión general de tool use**

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

Cada prompt de permiso es una llamada a tool por debajo.

- **Política de uso**

<https://www.anthropic.com/legal/usage-policy>

El uso empresarial sigue bajo la política de Anthropic, sea cual sea el modo.

- **Guardrails**

<https://docs.claude.com/en/docs/test-and-evaluate/strengthen-guardrails>

Hooks y listas allow/deny son el guardrail práctico cuando bypass está activo.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.